

Relatório Final: Geometria Computacional e Comida di Buteco

Davi Santos Rodrigues

Maio 2026

1 - Introdução e Definição do Problema

O presente trabalho prático teve como objetivo desenvolver uma aplicação de geoprocessamento em Python capaz de identificar e listar bares na cidade de Belo Horizonte que estejam situados dentro de um intervalo retangular de interesse.

O problema central consistiu em realizar buscas espaciais dado um endereço fornecido em formato de texto e o tamanho da diagonal geométrica da área delimitada, o sistema deve traduzir o endereço em coordenadas espaciais, calcular os limites do retângulo e varrer uma Kd tree para retornar os estabelecimentos contidos nessa área. Os resultados são apresentados em um mapa interativo e ordenados pela distância crescente em relação ao centro da busca.

2 - Decisões de Projeto e Arquitetura

O projeto buscou abranger as ferramentas requisitadas no enunciado do trabalho e a construção do sistema foi baseada na documentação dessas ferramentas e preferências pessoais:

- **Estrutura de Dados Principal:** Árvore K-Dimensional (*Kd Tree*) implementada nativamente conforme o material da disciplina.
- **Manipulação de dados:** Utilização do **DuckDB** para a ingestão dos dados brutos a partir de um arquivo CSV, transformando-os em objetos da classe *Boteco*. Para garantir a permanência e evitar redundância, uma vez coletado às coordenadas dos bares da lista inicial, escolhi armazenar os dados novamente em um csv.
- **Geocodificação Dinâmica:** Integração com a API do **Nominatim (OpenStreet-Map)** para converter a entrada de texto do usuário em coordenadas geográficas

lat/lon. A escolha dessa API é decorrente de buscas na internet, mais especificamente: dois vídeos no Youtube mostraram como é simples usar o Nominatim.

- **Aplicação Web e Mapa:** Biblioteca **Dash** combinada com **Dash-Leaflet**, adotada por sua capacidade de gerar componentes reativos web diretamente via código Python.

3 - Implementação da Estrutura de Dados: A Kd Tree

A *KdTree*, é responsável por particionar o espaço em duas dimensões e armazenar as coordenadas dos bares. Como o meu histórico de programação é principalmente C++, decidi implementar a estrutura utilizando Objetos e POO. A falta de encapsulamento, de ponteiros e de assinaturas de retorno, tornou mais complexo implementar a estrutura. E, devem existir, na minha implementação, "hábitos" do C++ que provavelmente não são os melhores em python.

A árvore particiona o espaço alternando os eixos de corte a cada nível de profundidade. Nos níveis pares, os nós são divididos pelo eixo x (Longitude); nos níveis ímpares, pelo eixo y (Latitude). Resolvi adotar a implementação vista em aula em que apenas as folhas armazenam os bares e que o corte é feito na mediana dos valores referentes ao eixo atual dos elementos da sub-árvore.

O algoritmo de busca recebe os limites do retângulo de interesse. A cada nó interno avaliado, o sistema verifica a intersecção geométrica. Caso a linha de corte do nó atual esteja inteiramente fora do retângulo delimitador, a árvore imediatamente realiza a poda de toda a sub-árvore correspondente. Ao atingir nós-folha, verifica se os mesmos pertencem ao intervalo desejado e constrói uma lista com os bares achados.

4 - Lógica Matemática e Tratamento Espacial

4.1 - Cálculo do Retângulo de Busca

Para definir a área de busca, optou-se por utilizar a forma de um quadrado. A partir da diagonal informada pelo usuário, a medida do lado (l) foi obtida pela relação geométrica direta:

$$l = \frac{diagonal}{\sqrt{2}}$$

O desafio seguinte foi converter essa distância de quilômetros para graus de latitude e longitude. A pesquisa inicial apontou o uso de uma constante padrão, assumindo que 1° geográfico equivale a aproximadamente 111 km.

Contudo, ao aplicar essa conversão uniforme de 111 km para ambos os eixos e plotar o polígono, a figura desenhada no mapa apresentava um formato retangular e alongado, diferente do quadrado planejado.

Pesquisando mais a fundo a causa desse problema visual, descobriu-se que a constante de 111 km é válida apenas para a latitude e a linha do equador. Como a Terra é esférica, a distância física correspondente a 1 grau de longitude "encolhe" à medida que nos afastamos do Equador em direção aos polos.

Para corrigir essa distorção e garantir que o polígono fosse um quadrado perfeito, foi necessário aplicar a aproximação equirretangular. Essa técnica ajusta a conversão da longitude multiplicando-a pelo cosseno da latitude central (ϕ) da área de busca:

$$\Delta\lambda_{\text{corrigido}} = \Delta\lambda_{\text{bruto}} \cdot \cos(\phi)$$

4.2 - Distância Real

Enquanto a Kd Tree realiza a filtragem e retorna os bares dentro do retângulo geométrico, o trabalho exige a apresentação do resultado ordenado crescentemente em relação à distância do endereço buscado. Para tal, o sistema calcula a distância real em linha reta entre as coordenadas centrais e cada bar utilizando a fórmula de **Haversine**, compensando a curvatura terrestre.

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (1)$$

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \quad (2)$$

$$d = R \cdot c \quad (3)$$

Onde:

- ϕ_1, λ_1 correspondem às coordenadas de origem;
- ϕ_2, λ_2 correspondem às coordenadas de destino;
- $\Delta\lambda$ e $\Delta\phi$ representam, respectivamente, a diferença de longitude e latitude entre os pontos;
- R é o raio médio da Terra (aproximadamente 6.371 km).

5 - Desenvolvimento da Interface e Dinâmica da Aplicação

A interface do sistema foi construída utilizando a biblioteca **Dash**, que permite a criação de aplicações web analíticas em Python integrando componentes HTML, CSS e JavaScript. Para a visualização geoespacial, utilizou-se o **Dash Leaflet**, cujos componentes foram adaptados a partir dos exemplos técnicos da documentação oficial da biblioteca.

5.1 - Inicialização

Ao ser inicializada, a aplicação realiza o *setup* do ambiente de dados antes de colocar o servidor em operação. Esse processo envolve:

- **Carga de Dados:** O arquivo `botecos_coordenadas.csv` é lido via DuckDB.
- **Construção da Estrutura:** A partir dos registros lidos, instanciam-se os objetos da classe *Boteco* e constrói-se a *Kd Tree* em memória para permitir buscas durante o tempo de execução.
- **Geometria dos Bairros:** É importado um arquivo **GeoJSON**, um formato padrão para representar estruturas geográficas simples em JSON e que contém a delimitação dos bairros de Belo Horizonte. No enunciado do trabalho, havia um link para uma discussão em um fórum sobre o formato das coordenadas disponibilizadas pela prefeitura de Belo Horizonte, formato este incompatível com o que é aceito pelo Dash Leaflet. Na mesma discussão havia um comentário com o arquivo em formato correto, portanto neste trabalho foi utilizado o arquivo disponibilizado no link: https://geoservicos.pbh.gov.br/geoserver/wfs?service=WFS&version=1.0.0&request=GetFeature&typeName=ide_bhgeo:BAIRRO&srsName=EPSG:4326&outputFormat=application%2Fjson.

5.2 - Lógica de Atualização e Callbacks

O funcionamento dinâmico da aplicação baseia-se em funções de **callback**. No Dash, um *callback* é acionado automaticamente sempre que uma propriedade de entrada, como o clique em um botão ou a digitação de um endereço, sofre alteração.

Durante o tempo de execução, o fluxo de operação segue esta ordem:

1. O usuário insere o endereço, a diagonal de busca e clica no botão.
2. O *callback* principal é disparado, acionando as funções auxiliares de geocodificação e cálculo do retângulo. Uma decisão de implementação foi concatenar o endereço digitado com “”, Belo Horizonte, Minas Gerais”, para que force a API a retornar somente endereços em Belo Horizonte.
3. A *Kd Tree* é consultada para filtrar os estabelecimentos.
4. A aplicação retorna uma lista de marcadores e o polígono de busca para serem renderizados no mapa e uma tabela que apresenta os bares encontrados na ordem requisitada pelo trabalho.

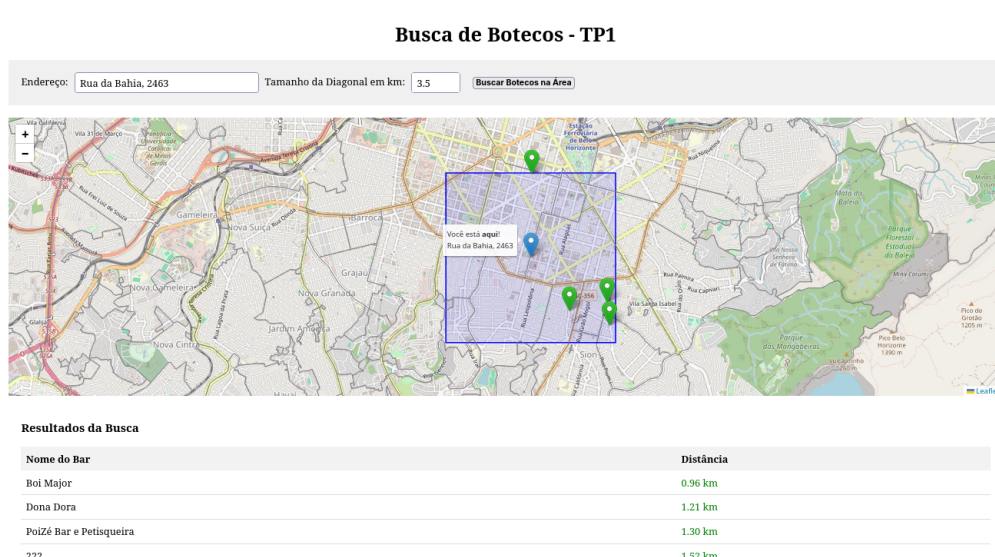


Figura 1: Exemplo de Busca

6 - Conclusão

A implementação demonstrou sucesso ao cumprir os requisitos estabelecidos para o projeto. A integração de uma estrutura de dados construída em baixo nível com frameworks modernos de visualização web resultou em uma aplicação funcional e pedagógica. O sistema realiza requisições geográficas em tempo real, poda árvores espaciais eficientemente e renderiza milhares de pontos georreferenciados de forma instantânea, consolidando os conceitos práticos de algoritmos de geometria computacional e interfaces interativas em Python.

Referências

- [1] VENESS, Chris. **Calculate distance, bearing and more between Latitude/Longitude points**. Movable Type Scripts, 2002-2022. Disponível em: <https://www.movable-type.co.uk/scripts/latlong.html>. Acesso em: 03 maio 2026.
- [2] MARQUES, Flávio. **Sistemas de Localização (LBS): Coordenadas Geográficas**. Teleco - Inteligência em Telecomunicações, 2007. Disponível em: https://teleco.com.br/tutoriais/tutorialsmsloc2/pagina_5.asp. Acesso em: 03 maio 2026.

- [3] DASH ONLINE. **Build a Real-Time Map with Dash Leaflet**. YouTube, 2021. Disponível em: <https://www.youtube.com/watch?v=H6hkSADi1Rc>. Acesso em: 03 maio 2026.
- [4] CHARMING DATA. **Dash Leaflet - Interactive Maps and Callbacks**. YouTube, 2021. Disponível em: <https://www.youtube.com/watch?v=rmIhGPy8rSY>. Acesso em: 03 maio 2026.